

# T-AIA-901

Travel Order Resolver

## **Groupe REN\_3**

Dorian AYOUL

Pierre MAUGER

Pierre HAMEL

Baptiste LEMONNIER

François PARMENTIER

# Table des matières

|                                     |    |
|-------------------------------------|----|
| Table des matières .....            | 2  |
| 1. Introduction.....                | 3  |
| 1.1 Contexte.....                   | 3  |
| 1.2 Objectifs Clients.....          | 4  |
| 1.3 Gestion de projet .....         | 5  |
| 2. Algorithmie .....                | 6  |
| 2.1 Initialisation .....            | 6  |
| 2.2 Transcription audio .....       | 7  |
| 2.3 Extraction des villes .....     | 8  |
| 2.4 Reconnaissance des villes ..... | 12 |
| 2.5 Dijkstra.....                   | 13 |
| 3. Stack Technique .....            | 15 |
| 3.1 Frontend.....                   | 15 |
| 3.2 Backend .....                   | 16 |
| 4. Conclusion.....                  | 17 |

# 1. Introduction

## 1.1 Contexte

Le projet « Travel Order Resolver » s'inscrit dans le cadre de l'option Intelligence artificielle (IA) en deuxième année de Master à Epitech.

Les étudiants en IA ont eu un projet introductif lors du semestre précédent avec la création d'un modèle de reconnaissance d'images sur le sujet classique des radiographies de poumons avec ou sans pneumonie.

Pour continuer la découverte de ce domaine, le sujet de « Travel order resolver » est proposé pour travailler entre autres sur le domaine de la reconnaissance de nom.

Le lien a été fait entre ce sujet et un précédent projet effectué à Epitech sous le nom d'« Epic Road Trip » qui avait pour but de proposer à un utilisateur de renseigner le départ et l'arrivée d'un itinéraire routier et mettre à disposition les activités, hébergements et restaurants disponibles à cette date.

Cependant, le lien entre ces deux projets s'est révélé peu évident lors de l'élaboration du projet et nous a même posé des problèmes au niveau technologiques, car nous avons développé l'API de l'Epic Road Trip en Golang, et que l'aspect IA du Travel Order Resolver nous a amené à proposer un backend uniquement en python (Cf. 3.2 – Stack technique - Backend) pour faciliter le travail.

Le résultat final nous a amené à faire une application avec deux fonctionnalités totalement différentes et non reliées. Un utilisateur arrivant sur la page d'accueil aura la possibilité d'accéder à « Epic Road Trip », l'ancienne application et sinon à « Epic Rail Trip », notre nouvelle application répondant au besoin client énoncé dans la partie suivante.

## 1.2 Objectifs Clients

Le projet présente des parcours utilisateurs assez simples et dont la difficulté ne réside pas dans l'optimisation de l'interface graphique mais plutôt dans la qualité et la précision de notre modèle.

Cependant, nous pouvons lister les objectifs d'expérience utilisateur suivants :

- Pouvoir aller d'une application à l'autre depuis la page d'accueil.
- Sur la partie Epic Rail Trip :
  - Arriver sur une interface minimaliste avec une carte centrée sur la France.
  - Pouvoir voir les performances globales du modèle utilisé.
  - Pouvoir lancer une recherche écrite depuis un champ texte.
  - Pouvoir lancer une recherche audio depuis un bouton.
  - Voir le tracé de l'itinéraire trouvé.
  - Voir les points de passage ainsi que les gares de correspondance.
  - Être notifié d'erreurs précises provenant du modèle en cas de problème.
  - Pouvoir relancer une recherche facilement.



*Schéma général du fonctionnement de l'application*

## 1.3 Gestion de projet

Nous nous sommes d'abord organisés grâce à l'outil Jira, où nous avons découpé l'intégralité des tâches et les avons affectées et quantifiées. Nous avons perdu l'intégralité de nos tickets dû à la fin de période d'essai du logiciel et avons dû trouver une solution plus pérenne de gestion de projets. Nous hébergeons désormais l'application open-source et gratuite Redmine sur un serveur DigitalOcean et nous n'avons donc plus le risque de perdre nos données.

✓ Appliquer ✕ Effacer 📄 Sauvegarder rapport personnalisé

| <input type="checkbox"/> | ▼ # | Tracker | Statut      | Priorité | Sujet                                                                  | Assigné à           | Mis-à-jour       |     |
|--------------------------|-----|---------|-------------|----------|------------------------------------------------------------------------|---------------------|------------------|-----|
| <input type="checkbox"/> | 22  | Task    | New         | Normal   | Send Reason why Algorithm failed - Model to API                        | Dorian Ayoul        | 09/01/2025 10:30 | ... |
| <input type="checkbox"/> | 21  | Task    | New         | Low      | Send Reason why Algorithm failed - API to Frontend                     | Baptiste Lemonnier  | 18/01/2025 19:01 | ... |
| <input type="checkbox"/> | 20  | Task    | Resolved    | Normal   | Add sentence patterns                                                  | Dorian Ayoul        | 18/01/2025 19:01 | ... |
| <input type="checkbox"/> | 19  | Task    | Resolved    | Normal   | Google slides - AIA                                                    | Pierre H            | 09/01/2025 17:25 | ... |
| <input type="checkbox"/> | 18  | Task    | Resolved    | Normal   | isCorrespondance - add to API                                          | Baptiste Lemonnier  | 17/01/2025 15:08 | ... |
| <input type="checkbox"/> | 17  | Task    | Resolved    | Normal   | IsCorrespondance + Correct duration between stations - Send to Backend | Pierre Mauger       | 17/01/2025 15:07 | ... |
| <input type="checkbox"/> | 16  | Task    | Resolved    | Urgent   | Mise en forme Marker Front                                             | François PARMENTIER | 17/01/2025 17:15 | ... |
| <input type="checkbox"/> | 15  | Bug     | Resolved    | Urgent   | List Stations : Last station + Correct duration + isCorrespondance     | François PARMENTIER | 17/01/2025 17:15 | ... |
| <input type="checkbox"/> | 14  | Task    | New         | Normal   | Modale Marker                                                          | François PARMENTIER | 09/01/2025 10:03 | ... |
| <input type="checkbox"/> | 13  | Task    | Resolved    | Normal   | Error management - Frontend                                            | François PARMENTIER | 16/01/2025 17:06 | ... |
| <input type="checkbox"/> | 12  | Task    | Resolved    | Normal   | Error management - Backend                                             | Baptiste Lemonnier  | 17/01/2025 15:08 | ... |
| <input type="checkbox"/> | 11  | Bug     | Resolved    | High     | Correct city matching                                                  | Baptiste Lemonnier  | 10/01/2025 10:16 | ... |
| <input type="checkbox"/> | 10  | Task    | Resolved    | Normal   | Report - BaptisteLEMONNIER                                             | Baptiste Lemonnier  | 17/01/2025 18:27 | ... |
| <input type="checkbox"/> | 9   | Task    | Resolved    | Normal   | Report - DorianAYOUL                                                   | Dorian Ayoul        | 17/01/2025 16:35 | ... |
| <input type="checkbox"/> | 8   | Task    | New         | Normal   | Report - PierreHAMEL                                                   | Pierre H            | 09/01/2025 09:50 | ... |
| <input type="checkbox"/> | 7   | Task    | In Progress | Normal   | Report - FrancoisPARMENTIER                                            | François PARMENTIER | 09/01/2025 17:19 | ... |
| <input type="checkbox"/> | 6   | Task    | New         | Normal   | Report - PierreMAUGER                                                  | Pierre Mauger       | 09/01/2025 09:49 | ... |

*La liste de tickets qui a permis une gestion de projets rapide et claire en temps réel*

Notre serveur Redmine est accessible à l'adresse : <http://138.68.71.72:10083/>

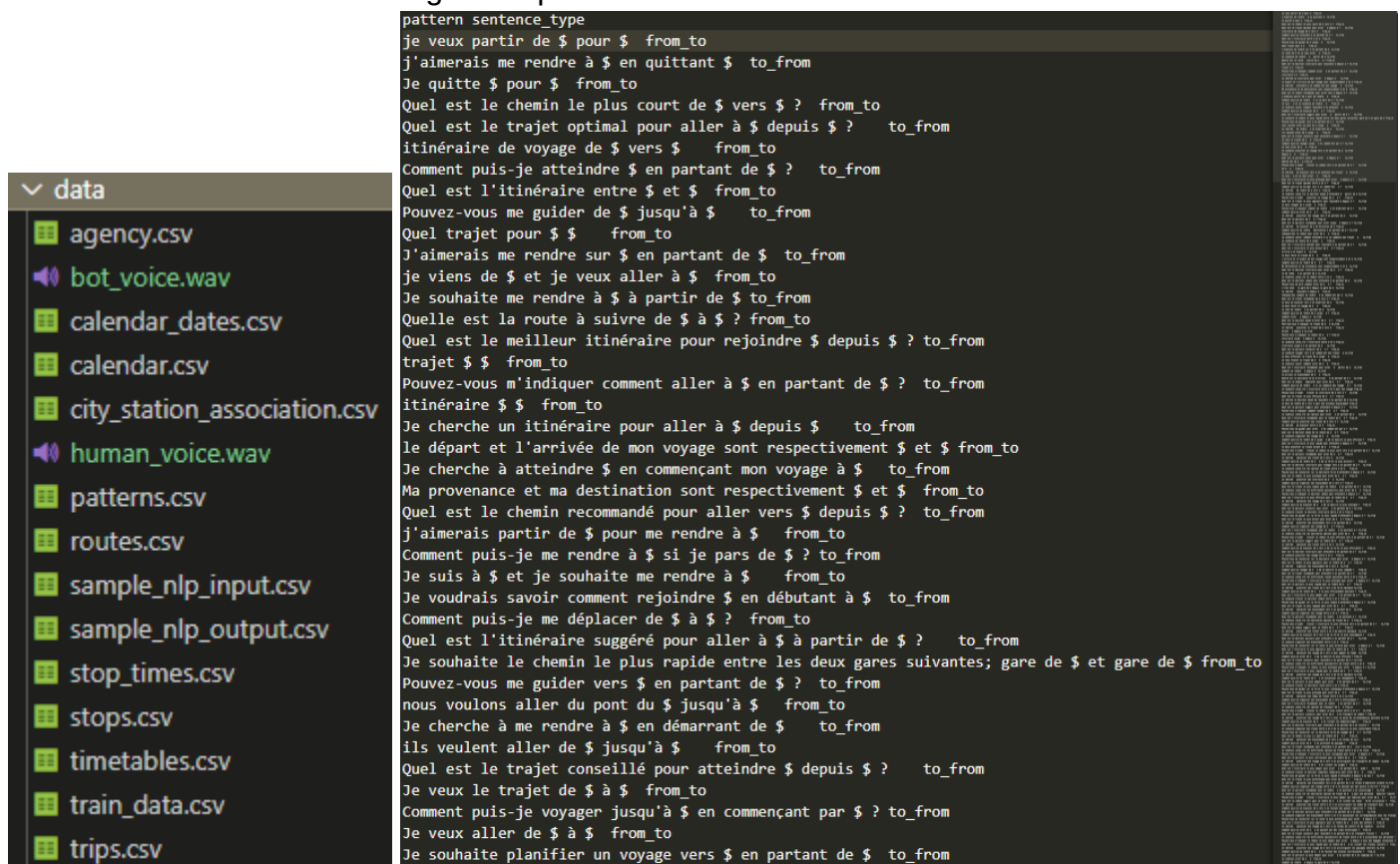
## 2. Algorithmie

### 2.1 Initialisation

Tout bon projet de [machine learning](#) commence par une création et un nettoyage rigoureux de la donnée en amont.

Nous avons tout d'abord utilisé un script pour convertir les fichiers fournis par Epitech contenant les données de la SNCF, qui étaient au format .txt au lieu de .csv malgré des données structurées en CSV, et pour supprimer les colonnes inutilisées afin de clarifier leurs futures analyses.

Afin d'entraîner un modèle capable de reconnaître les paternes de phrases de trajet dans les entrées, il est également nécessaire de disposer de données étiquetées comprenant plusieurs façons de demander un trajet, ainsi que le type de phrase associé (ville de départ ou ville d'arrivée mentionnée en premier). Notre script lit un fichier contenant des schémas de phrases de trajet (imaginé par nous-même et ChatGPT) et une liste de villes (tirées de la SNCF). Pour chaque paterne, il génère 100 phrases en remplaçant des espaces réservés par des paires de villes aléatoires et associe chaque phrase générée à son type. Les résultats sont ensuite sauvegardés pour être utilisés dans l'entraînement du modèle.



*Données tabléées de la SNCF nettoyées/converties en CSV et paternes de phrases de trajets utilisés pour la création d'une base de données d'entrainement*

## 2.2 Transcription audio

Le but premier du projet étant de traiter des données d'entrée sous format textuel, cette partie optionnelle de transcription audio vers texte ne devait pas prendre trop de temps.

Ce programme est conçu pour effectuer la transcription d'audio en français, en utilisant la bibliothèque [speech\\_recognition](#) couplée à [PyAudio](#) pour capturer l'audio en temps réel via un microphone ou à partir d'un fichier audio au format `.wav`. L'outil s'appuie sur les capacités de [Google Speech Recognition](#) pour convertir l'audio en texte, offrant ainsi une reconnaissance vocale fiable et adaptée à la langue française.

L'audio peut être traité de deux manières : soit à partir d'un fichier déjà existant, soit en enregistrant directement depuis le microphone, avec une calibration automatique pour s'adapter aux bruits ambiants. Une fois l'audio capté, il est envoyé au moteur de reconnaissance vocale qui retourne la transcription. Le programme gère également les erreurs courantes, comme les problèmes de connexion au service ou les cas où l'audio n'est pas compréhensible, garantissant une exécution fluide et robuste.

La transcription obtenue est ensuite ajoutée au fichier CSV utilisé par notre modèle d'extraction des villes. Si le fichier n'existe pas encore, il est automatiquement créé, assurant ainsi une gestion transparente et cohérente des transcriptions.

```
append_to_csv(transcribe_audio("./data/human_voice.wav"))  
✓ 0.9s  
  
Recording ./data/human_voice.wav  
Transcription: je veux aller de Paris à Rennes  
Opening/creating ./nlp_input.csv  
Successfully saved the transcription !
```

*Bonne transcription en enregistrement d'un fichier audio `.wav` fourni au script*

## 2.3 Extraction des villes

Ce programme est une solution avancée pour extraire les villes de départ et d'arrivée sur les phrases de trajet en français, comme celles obtenues par notre outil de transcription audio.

Le processus commence par la lecture des phrases et de leurs identifiants depuis le fichier CSV d'entrée. Chaque phrase est ensuite analysée pour déterminer si elle contient une description de trajet valide. Si la phrase n'est pas détectée comme française (via la librairie [langdetect](#)) ou si elle ne contient pas exactement deux villes reconnues dans la base de données des villes prises en charge (issues des fichiers SNCF), elle est classée comme non valide ou inconnue.

```
1 je voudrais aller de Toulouse à bordeaux
2 Comment me rendre à Port-Boulet depuis la gare de Tours ?
3 Je veux aller voir mon ami Albert à Tours en partant de Bordeaux
4 Il y a-t-il des trains de Nantes à Montaigu
5 Une phrase sans origine ni destination
Si pas de numéro de séquence, on considère que c'est zéro
7 is there any train going from Paris to Marseille ?
8 Je veux aller de Rennes à Paris en passant par Caen
```

### *Phrases d'entrée*

```
1 Toulouse Bordeaux
2 Tours Port-Boulet
3 Bordeaux Tours
4 Nantes Montaigu
5 NOT_TRIP
6 NOT_TRIP
7 NOT_FRENCH
8 UNKNOWN
```

### *Résultats de sortie*

Lorsque la phrase est valide, le NER ([Named Entity Recognition](#)) identifie les lieux. [NERmembert](#) est une version calibrée de [CamemBERT](#) pour la tâche de NER, qui consiste à identifier les entités dans la langue française, et plus précisément dans notre cas les LOC (LOCations, comme les villes).

Celui-ci a été entraîné par [CATIE](#) sur le jeu de données françaises concaténé et nettoyé [frenchNER\\_3entities](#) (420 264 lignes, dont 346 071 avec 303 722 LOC utilisées pour l'entraînement, 32 951 avec 30 279 LOC pour la validation et 41 242 avec 39 195 LOC pour les tests). Il se compare favorablement à d'autres modèles NER français similaires.

| Model                                                   | PER          | LOC          | ORG          |
|---------------------------------------------------------|--------------|--------------|--------------|
| <a href="#"><u>Jean-Baptiste/camembert-ner</u></a>      | 0.941        | 0.883        | 0.658        |
| <a href="#"><u>cmarkea/distilcamembert-base-ner</u></a> | 0.942        | 0.882        | 0.647        |
| <a href="#"><u>NERmembert-base-3entities</u></a>        | 0.966        | 0.940        | 0.876        |
| NERmembert-large-3entities (this model)                 | <b>0.969</b> | <b>0.947</b> | <b>0.890</b> |
| <a href="#"><u>NERmembert-base-4entities</u></a>        | 0.951        | 0.894        | 0.671        |
| <a href="#"><u>NERmembert-large-4entities</u></a>       | 0.958        | 0.901        | 0.685        |

*Comparaison de précision des résultats pour la détection des entités sur frenchNER\_3entities par rapport à des modèles similaires, LOC est ce qui nous intéresse*

Il a été construit avec [PyTorch](#) et les paramètres suivants :

- learning\_rate : 2e-05 (taux d'apprentissage, contrôle la vitesse de mise à jour des poids du modèle pendant l'entraînement)
- train\_batch\_size : 8 (taille du lot d'échantillons utilisé pour entraîner le modèle)
- eval\_batch\_size : 8 (taille du lot d'échantillons utilisé pour évaluer le modèle)
- seed : 42 (graine d'aléatoire fixe pour garantir la reproductibilité des résultats)
- optimizer : Adam avec betas=(0.9, 0.999) et epsilon=1e-08 (algorithme de mise à jour des poids)
- lr\_scheduler\_type : linéaire (réduction progressive du taux d'apprentissage pendant l'entraînement)
- num\_epochs : 3 (nombre total de passages sur l'ensemble des données d'entraînement)

Le modèle a été entraîné avec [l'impact environnemental estimé](#) suivant (basé sur le type de matériel, le temps d'exécution, le fournisseur cloud et la région de calcul) :

- Carte graphique : A100 PCIe 40/80GB
- Heures totales : 4h31min
- Fournisseur cloud : Infrastructure privée

- Efficacité carbone (kg/kWh) : 0,077 (estimée via [electricitymaps](#) pour le 12 janvier 2024)
- Émissions carbone (consommation énergétique x temps x production de carbone basée sur la région du réseau électrique) : 0,009 kg eq. CO2

À noter que nous avons initialement opté pour le modèle [fr\\_core\\_news\\_lg](#) de [spaCy](#). Cependant, après avoir évalué la précision de détection des deux villes dans chaque phrase de nos données d'entraînement à l'aide d'un script, nous avons constaté que ce modèle atteignait une précision de 79,31%. Cette performance, souvent impactée par des cas ambigus (comme "Albert", pouvant être à la fois un prénom et une ville) ou par des villes inconnues du modèle, était nettement inférieure à celle de NERmembert qui obtenait 89,7% et vers lequel on s'est donc orientés.

Une fois NERmembert et les CSV des données SNCF chargés, les données d'entraînement du classificateur sont, si elles existent, réutilisées ou créées grâce à des fichiers [Joblib](#), évitant ainsi de réentraîner le modèle à chaque exécution.

L'algorithme de traitement lui-même utilise des outils tels que le [CountVectorizer](#), le [TfidfTransformer](#) et le modèle [Multinomial Naive Bayes](#). Le CountVectorizer convertit les phrases en une représentation numérique sous forme de matrice de fréquences des mots, tandis que le TfidfTransformer applique une pondération pour mettre en avant les mots rares mais significatifs, réduisant ainsi l'impact des mots courants. Ces vecteurs transformés servent ensuite d'entrée pour le classificateur Naive Bayes qui prédit avec précision le type de trajet ("from\_to" (aller) ou "to\_from" (retour)) en le comparant aux nombreux exemples de patronnes de phrases envisageables.

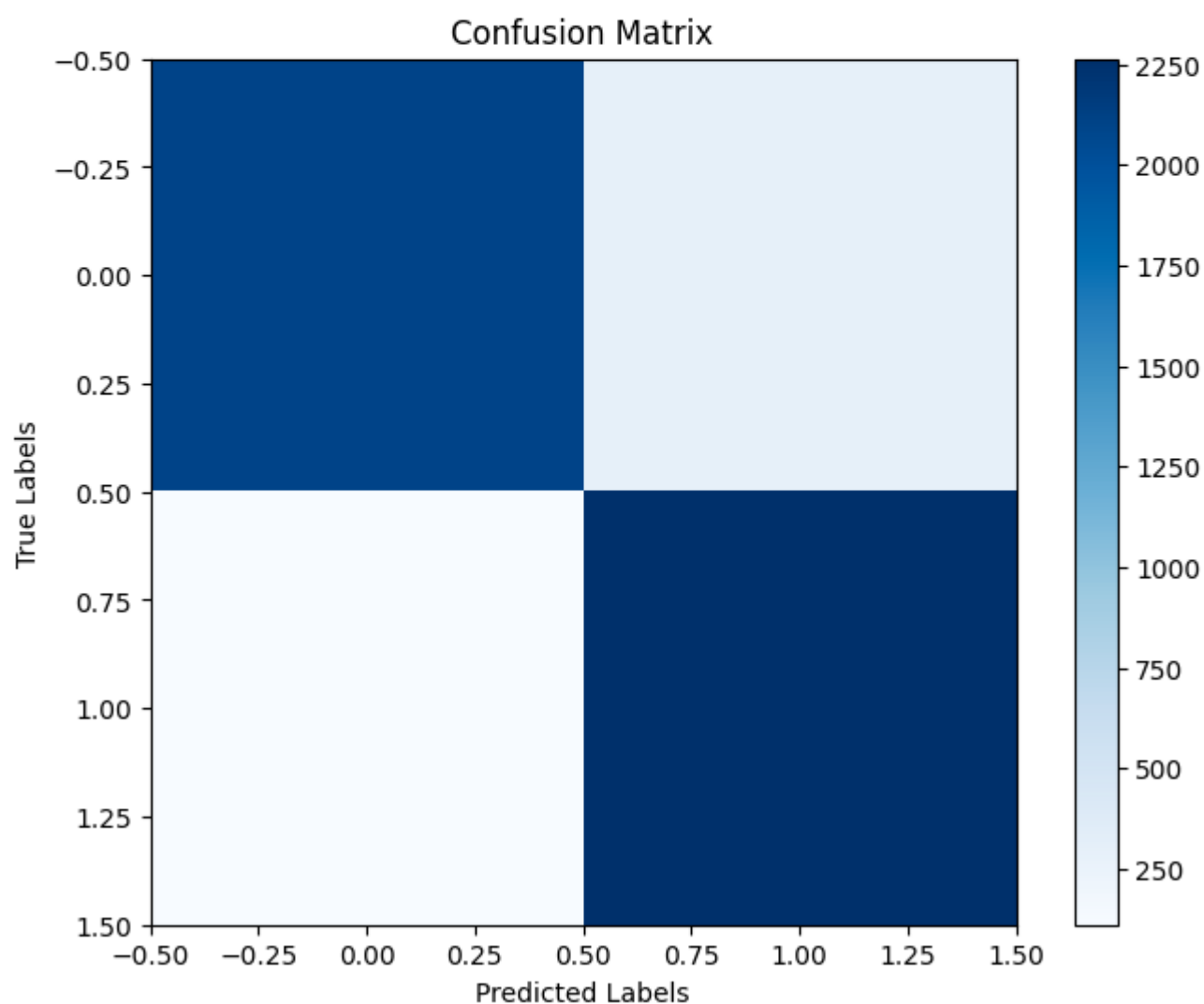
Une fois la courbe du classificateur ajustée à la détection, celle-ci est utilisée afin de prédire le sens de la phrase de trajet valide entrée par l'utilisateur.

Les résultats sont particulièrement satisfaisants, avec une exactitude totale de 92%. Le rapport de classification affiche une précision équilibrée entre les deux classes principales : 91% pour "from\_to" et 92% pour "to\_from", démontrant un bon équilibre entre précision et rappel (justesse des prédictions positives et capacité à trouver tous les vrais positifs). La matrice de confusion confirme que la majorité des phrases sont classées correctement, avec seulement une faible proportion d'erreurs.

Accuracy: 0.92

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| from_to      | 0.95      | 0.88   | 0.91     | 2386    |
| to_from      | 0.89      | 0.95   | 0.92     | 2374    |
| accuracy     |           |        | 0.92     | 4760    |
| macro avg    | 0.92      | 0.92   | 0.92     | 4760    |
| weighted avg | 0.92      | 0.92   | 0.92     | 4760    |



Enfin, les résultats sont écrits dans un fichier CSV de sortie. Chaque ligne de ce fichier contient l'identifiant de la phrase, une classification du type de phrase (trajet valide, non valide, ou non français), et les villes de départ et d'arrivée dans l'ordre correct si un trajet est détecté.

## 2.4 Reconnaissance des villes

Lors de la saisie d'une ville, des erreurs de frappe ou des variantes de noms peuvent empêcher de trouver une correspondance exacte avec notre base de données. Pour garantir que l'algorithme de calcul d'itinéraire (Dijkstra) fonctionne correctement, il est essentiel de normaliser et corriger les noms de villes pour qu'ils correspondent à des données existantes.

La première étape consiste à réduire la liste des villes candidates en ne conservant que celles dont la première lettre correspond à celle de l'entrée utilisateur. Cette approche simple est efficace, car il est rare que l'utilisateur se trompe sur la première lettre.

- Exemple : Pour "Marseille", seules les villes commençant par "M" sont conservées.

Nous comparons chaque ville restante avec l'entrée utilisateur en calculant un score basé sur les caractères successifs communs. Ce score représente la proportion de lettres alignées correctement.

- Exemple : Pour "Marseille", la précision avec "Marseille" est de 88%.

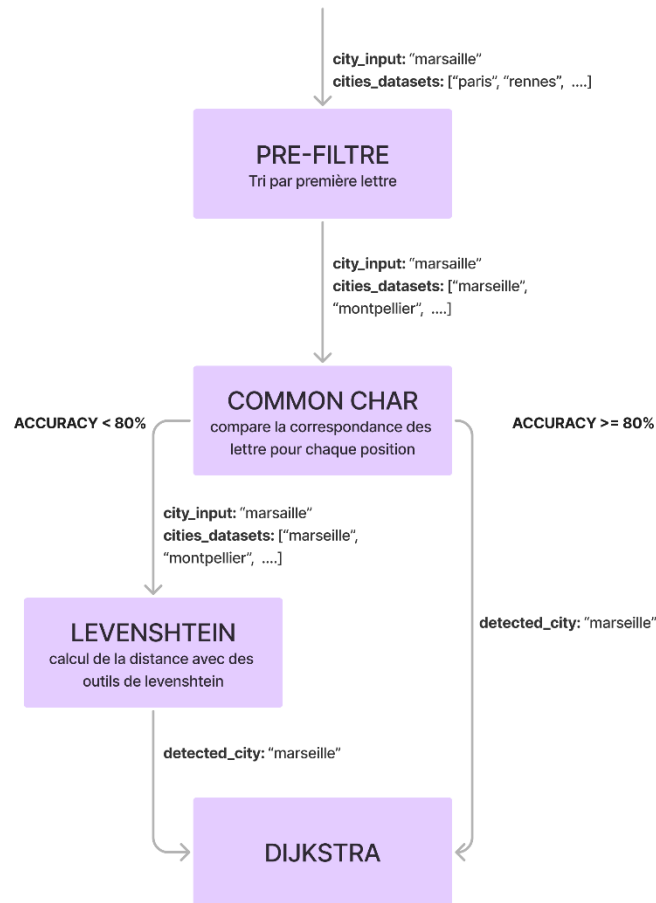
En fonction des scores obtenus, trois cas peuvent se présenter :

- Une seule ville avec un score  $\geq 80\%$  : La ville est directement retournée comme résultat.
- Plusieurs villes avec un score  $\geq 80\%$  : Une ambiguïté est détectée, et nous demandons à l'utilisateur de choisir parmi les options proposées.
- Aucune ville avec un score  $\geq 80\%$  : Nous passons à une méthode plus avancée.

### 4. Distance de Levenshtein

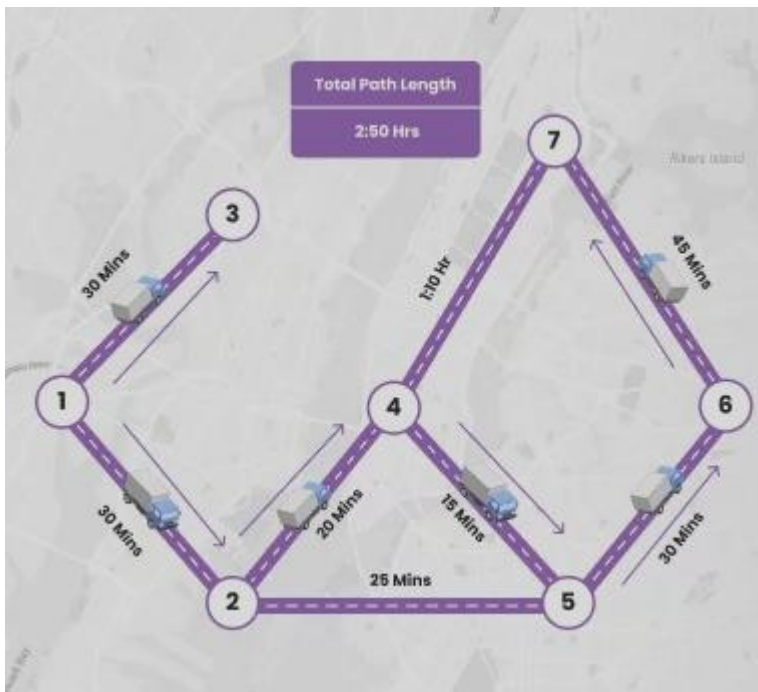
Si aucune correspondance claire n'est trouvée, nous utilisons la distance de Levenshtein, qui mesure le nombre de modifications nécessaires pour transformer un mot en un autre.

Nous combinons également cette approche avec le « matching blocks », qui identifie les blocs de caractères consécutifs similaires sans se limiter aux positions exactes. Cela permet une comparaison plus sophistiquée et tolérante aux erreurs.



## 2.5 Dijkstra

Pour résoudre le problème du pathfinding, nous avons opté pour l'algorithme de Dijkstra, implémenté en Python, afin de déterminer les trajets les plus rapides entre deux gares. Ce choix s'explique par la nature du problème, qui consiste à identifier les chemins optimaux dans un graphe pondéré où les gares sont représentées comme des nœuds et les trajets ferroviaires comme des arêtes, pondérées selon le temps de trajet. L'algorithme de Dijkstra s'est révélé particulièrement adapté grâce à sa capacité à calculer efficacement les plus courts chemins à partir d'un point source tout en tenant compte des pondérations.



Pour alimenter le graphe, nous avons analysé des documents contenant les horaires des trains et les lignes reliant les différentes gares. Ces données ont permis de définir les poids des arêtes en fonction du temps de trajet. Une difficulté résidait dans la correspondance entre les noms des gares et leurs localisations géographiques. Nous avons donc exploité l'API SNCF pour obtenir une liste exhaustive des gares, incluant leurs données géographiques, que nous avons ensuite consolidées dans un fichier CSV pour une utilisation ultérieure.

Python s'est imposé comme un choix naturel pour ce projet en raison de sa compatibilité avec notre architecture existante, principalement développée en Python. La bibliothèque pandas a été utilisée pour traiter les fichiers CSV efficacement, tandis que heapq et defaultdict ont permis d'optimiser la gestion des structures de données et de réduire les temps d'exécution lors de la construction et de l'exploitation du graphe.

## 3. Stack Technique

### 3.1 Frontend

Comme expliqué dans le chapitre Contexte de notre introduction (Cf 1.1), nous avons essayé de suivre les directives d'Epitech quant à l'intégration du Travel Order Resolver (TOR) au sein de notre Epic Road Trip (ERT). Cependant, nous avons découvert avec le temps que les deux projets présentaient des caractéristiques très différentes (notamment l'utilisation de l'IA nous incitant d'utiliser Python sur toute la stack backend) et que nous ne pouvions pas si facilement utiliser les composants faits spécifiquement pour l'ERT.

Avant de nous rendre compte de ce problème, nous avons repris la stack frontend du précédent projet fait avec le Framework ReactJS, outil particulièrement pratique pour l'interaction avec une carte fournie par l'API de Google Maps. Nous avons utilisé cette API car elle présente des fonctionnalités exhaustives pour la bonne résolution du projet :

- Une carte détaillée et navigable.
- La possibilité de tracer des itinéraires entre deux destinations avec huit étapes disponibles.
- Un outil de Geocoding (transformation de point d'intérêt en latitude / longitude) intégré pour placer les marqueurs sur la carte.

Les défis en frontend ont été les suivants :

- La reprise et la transformation des objets existants sur l'ERT pour pouvoir les réutiliser dans le TOR.
- La captation et l'envoi de l'audio vers le backend pour l'analyse.
- La gestion des erreurs venant du backend.
- L'adaptation des itinéraires et étapes renvoyées par le backend avec leurs propriétés (correspondance, durée, distance) avec l'API de Google Maps dont nous étions tributaires.

## 3.2 Backend

Nous avons choisi Python pour le backend car c'est le langage de référence lorsqu'on travaille sur des projets d'intelligence artificielle. Il est largement utilisé pour l'IA et le traitement de données grâce à une bibliothèque de package puissant, nombreux et bien adaptées.

Pour l'API, nous avons utilisés FastAPI. C'est un framework rapide et moderne qui facilite la création d'APIs RESTful. FastAPI offre une bonne gestion des requêtes asynchrones, ce qui est important pour assurer de bonnes performances.

Uvicorn comme serveur pour l'application. Uvicorn est un serveur ASGI qui permet de gérer les requêtes de manière asynchrone, ce qui est crucial pour maintenir de bonnes performances, même quand l'application reçoit de nombreuses requêtes. Il est léger et rapide, et s'intègre parfaitement avec FastAPI, ce qui permet de déployer l'application de manière fluide et rapide.

## 4. Conclusion

Le projet *Travel Order Resolver* a permis d'explorer des concepts fondamentaux de l'intelligence artificielle appliquée au traitement du langage naturel. En intégrant des technologies externes comme NERmembert pour la reconnaissance d'entités géographiques et internes telles que l'analyse de fréquence linguale et la classification naïve bayésienne pour l'ordre de trajet, nous avons atteint des résultats convaincants avec une précision globale de 92%.

L'application propose une interface fonctionnelle permettant à l'utilisateur d'effectuer des recherches textuelles ou vocales pour générer des itinéraires ferroviaires en France. Les données utilisées, nettoyées et enrichies, ont renforcé la fiabilité des prédictions, tandis que l'architecture technique basée sur Python et React garantit la modularité et l'efficacité du système.

Malgré des défis liés à l'intégration avec des projets précédents, le *Travel Order Resolver* démontre la faisabilité d'une solution robuste et évolutive pour répondre à des cas d'usage concrets dans le domaine des transports. Les performances du modèle et les enseignements tirés du projet constituent une base solide pour des itérations futures.